

```
In [1]: def greet():  
        print('hello')  
        print('good morning team')
```

```
In [2]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()
```

```
hello  
good morning team
```

```
In [6]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        def greet():  
            print('hello')  
            print('good morning team')  
            greet()
```

```
hello  
good morning team  
hello  
good morning team
```

```
In [7]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        print()  
  
        def greet():  
            print('hello')  
            print('good morning team')  
            greet()
```

```
hello  
good morning team  
  
hello  
good morning team
```

```
In [8]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        print()  
  
        def greet():  
            print('hello')  
            print('good morning team')
```

```

greet()

print()

def greet():
    print('hello')
    print('good morning team')
greet()

```

hello
good morning team

hello
good morning team

hello
good morning team

```

In [9]: def greet(): # declare function without argument
        print('hello')
        print('good morning team')
        greet()
        print('*****')
        greet()
        print('*****')
        greet() # function calling with out argument)

```

hello
good morning team

hello
good morning team

hello
good morning team

```

In [10]: # function without argument

```

```

def greet():
    print('hello')
    print('good morning team')
greet()

```

hello
good morning team

```

In [11]: # function with argument

```

```

def add(x,y):
    c=x+y
    print(c)

add(5,6)

```

11

```

In [12]: # function with argument

```

```
def add(x,y):  
    c=x+y  
    return c  
  
add(5,6)
```

Out[12]: 11

In [13]: *# function with argument*

```
def add(x,y):  
    c=x+y  
    return c  
  
add(5)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[13], line 7  
      4     c=x+y  
      5     return c  
----> 7 add(5)  
  
TypeError: add() missing 1 required positional argument: 'y'
```

In [14]: *# function with argument*

```
def add(x,y):  
    c=x+y  
    return c  
  
add(5,6,7)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[14], line 7  
      4     c=x+y  
      5     return c  
----> 7 add(5,6,7)  
  
TypeError: add() takes 2 positional arguments but 3 were given
```

In [15]: *# function with argument*

```
def add(x,y,z):  
    c=x+y  
    return c  
  
add(5,6,7)
```

Out[15]: 11

In [16]: *# function with argument*

```
def add(x,y,z):  
    c=x+y+z+m
```

```

    return c

add(5,6,7)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[16], line 7
      4     c=x+y+z+m
      5     return c
----> 7 add(5,6,7)

Cell In[16], line 4, in add(x, y, z)
      3 def add(x,y,z):
----> 4     c=x+y+z+m
      5     return c

NameError: name 'm' is not defined

```

In [17]: *# function with argument*

```

def add(x,y,z,n):
    c = x+y+z+m
    return c

add(5,6,7,8)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[17], line 7
      4     c = x+y+z+m
      5     return c
----> 7 add(5,6,7,8)

Cell In[17], line 4, in add(x, y, z, n)
      3 def add(x,y,z,n):
----> 4     c = x+y+z+m
      5     return c

NameError: name 'm' is not defined

```

In [18]: *# function with argument*

```

def add(x,y,z,n):
    c = x+y+z+n
    return c

add(5,6,7,8)

```

Out[18]: 26

```

In [19]: def greet():
          print('hello')
          print('good morning team')
          greet()

          def add(x,y):

```

```
c = x+y  
return c  
  
add(5,6)
```

```
hello  
good morning team
```

Out[19]: 11

```
In [20]: def greet():  
          print('hello')  
          print('good morning team')  
  
          def add(x,y):  
              c = x+y  
              return c  
  
          def sub(x,y):  
              d = x-y  
              return d  
  
          greet()  
          print(add(5,6))  
          print(sub(5,6))
```

```
hello  
good morning team  
11  
-1
```

```
In [21]: def add_sub(x,y):  
          c= x+y  
          d= x-y  
          return c, d  
  
          result = add_sub(4,5)  
          print(result)  
          print(type(result))
```

```
(9, -1)  
<class 'tuple'>
```

```
In [22]: def add_sub(x,y):  
          c= x+y  
          d= x-y  
          return c, d  
  
          result, result1 = add_sub(4,5)  
  
          print(result)  
          print(result1)  
          print(type(result))
```

```
9  
-1  
<class 'int'>
```

```
In [23]: def add_sub_mul(x,y):
          c= x+y
          d= x-y
          e= x*y
          return c, d, e

          add, sub, mul = add_sub_mul(4,5)

          add
          sub
          mul
```

Out[23]: 20

Update

```
In [24]: def update():
          x = 8
          print(x)
          update()
```

8

```
In [25]: def update(): #update function take the value from the user
          x = 8
          print(x)

          update(8)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[25], line 5
      2     x = 8
      3     print(x)
----> 5 update(8)

TypeError: update() takes 0 positional arguments but 1 was given
```

```
In [26]: def update(x): #update function take the value from the user
          x = 8
          return x

          update(100)
```

Out[26]: 8

```
In [27]: def update(x):
          x = 8
          return x

          a = 15
          update(a)
          print(a)
```

15

7th

```
In [28]: def person(name,age):  
         print(name)  
         print(age)  
  
         person('nit', 22)
```

nit
22

```
In [29]: def person(name,age):  
         print(name)  
         print(age)  
  
         person('nit')
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[29], line 5  
      2     print(name)  
      3     print(age)  
----> 5 person('nit')  
  
TypeError: person() missing 1 required positional argument: 'age'
```

```
In [30]: def person(name,age):  
         print(name)  
         print(age)  
  
         person(22)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[30], line 5  
      2     print(name)  
      3     print(age)  
----> 5 person(22)  
  
TypeError: person() missing 1 required positional argument: 'age'
```

```
In [31]: def person(name,age):  
         print(name)  
         print(age)  
  
         person('nit', 22, 23, 45, 56)
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[31], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit', 22, 23, 45, 56)

TypeError: person() takes 2 positional arguments but 5 were given

```

```

In [32]: def person(name, age):
          print(name)
          print(age)

          person(22, 'nit')

```

```

22
nit

```

```

In [33]: def person(name, age):
          print(name)
          print(age-1)

          person(22, 'nit')

```

```

22

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[33], line 5
      2     print(name)
      3     print(age-1)
----> 5 person(22, 'nit')

Cell In[33], line 3, in person(name, age)
      1 def person(name, age):
      2     print(name)
----> 3     print(age-1)

TypeError: unsupported operand type(s) for -: 'str' and 'int'

```

```

In [34]: def person(name, age):
          print(name)
          print(age-1)

          person(age = 22, name = 'nit')

```

```

nit
21

```

```

In [35]: def person(name, age, new_age ):
          print(name)
          print(age-1)

          person(age = 22, name = 'nit')

```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[35], line 5  
      2     print(name)  
      3     print(age-1)  
----> 5 person(age = 22, name = 'nit')  
  
TypeError: person() missing 1 required positional argument: 'new_age'
```

```
In [36]: def person(name, age, new_age ):  
        print(name)  
        print(age-1)  
        print(new_age)  
  
        person(age = 22, name = 'nit', new_age=23)
```

```
nit  
21  
23
```

Default Argument

```
In [37]: def person(name, age=18):  
        print(name)  
        print(age)  
  
        person('nit')
```

```
nit  
18
```

```
In [38]: def person(name, age=18):  
        print(name)  
        print(age)  
  
        person('nit', 40)
```

```
nit  
40
```

Variable Length Argument

```
In [39]: def person(name, age):  
        print(name)  
        print(age)  
  
        person('nit', 40, 50, 60, 70, 80 )
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[39], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit', 40, 50, 60, 70, 80 )

TypeError: person() takes 2 positional arguments but 6 were given
```

```
In [40]: def sum(a, b):
          c = a+b
          print(c)

          sum(5,6,7,8)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[40], line 5
      2     c = a+b
      3     print(c)
----> 5 sum(5,6,7,8)

TypeError: sum() takes 2 positional arguments but 4 were given
```

```
In [41]: def sum(a, *b):
          c = a+b
          print(c)

          sum(5,6,7,8)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[41], line 5
      2     c = a+b
      3     print(c)
----> 5 sum(5,6,7,8)

Cell In[41], line 2, in sum(a, *b)
      1 def sum(a, *b):
----> 2     c = a+b
      3     print(c)

TypeError: unsupported operand type(s) for +: 'int' and 'tuple'
```

```
In [42]: def sum(a, *b): # 1st argument is fixed but for 2nd argument
          #c = a+b
          print(type(a))
          print(type(b))

          sum(5,6,7,8)
```

```
<class 'int'>
<class 'tuple'>
```

```
In [43]: def sum(a, *b): # 1st argument is fixed & we fetch each value from the tuple & we c
          c = a
```

```

    for i in b:
        c = c + i
    print(c)

sum(5,6,7,8)

```

26

```

In [44]: def sum(a, *b): # 1st argument is fixed & we fetch each value from the tuple & we c
        c = a
        for i in b:
            c = c + i
        print(c)

sum(5,6,7,8,9,20)

```

55

```

In [45]: def sum(a, *b): # 1st argument is fixed & we fetch each value from the tuple & we c
        c = a
        for i in b:
            c = c + i
        print(c)

sum(5,6,7,8)

```

26

Kwargs

```

In [46]: def person():
        person('ALEX', 36, 'JOHN', 987767)

```

```

In [47]: def person(name, *data):
        print(name)
        print(data)

person('ALEX', 36, 'JOHN', 987767)

```

ALEX
(36, 'JOHN', 987767)

```

In [48]: def person(name,*data):
        print('name')
        print(data)

person('ALEX', age = 36, home_place ='southcity', mob =987767)

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[48], line 5
      2     print('name')
      3     print(data)
----> 5 person('ALEX', age = 36, home_place = 'southcity', mob = 987767)

TypeError: person() got an unexpected keyword argument 'age'

```

```

In [49]: def person(name, **data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place = 'southcity', mob = 987767)

```

```

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767}

```

```

In [50]: def person(name, **data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place = 'southcity', mob = 987767, salary = 40000, married = 'yes')

```

```

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767, 'salary': 40000, 'married': 'yes'}

```

Functional Argument we are completed

global variable & local variable

```

In [51]: a = 10  #-- global variable

          def something():
              b = 15 #local variable

```

```

In [52]: a = 10  #-- global variable

          def something():
              b = 15 #local variable

              print('in function',b)
              print('out function',a)

```

```

In [53]: a = 10
          def something():
              b = 15
              print('in function',b)

          print('out function',a)

```

out function 10

```
In [54]: a = 10

def something():
    a = 15

print('in function',a)

print('out function',a)
```

in function 10
out function 10

```
In [55]: a = 10

def something():
    b = 15
    print('in function',b) # local variable
    something()

print('out function',a) # gloabl variable
```

in function 15
out function 10

```
In [56]: a = 10

def something():
    #if we remove this variable then can befault it consider as global variable
    print('in function',a)

something()
print('out function',a)
```

in function 10
out function 10

```
In [57]: a = 10
def something():
    global a
    b = 15 # 15 is converted to local when user assigned global a
    print('in function',b)
    print('gloabl variable', a)

something()

print('out function',a)
```

in function 15
gloabl variable 10
out function 10

```
In [58]: a = 20

def something():
    global a
    a = 15
```

```
print('in function',a)

a = 15

something()
print('out function',a)
```

in function 15
out function 15

```
In [59]: x = 10 # Global variable

def update_x():
    global x # Declare that we are using the global variable x
    x += 5 # Modify the global variable

update_x()
print(x) # Output: 15
```

15

```
In [60]: x = 10 # Global variable

def update_x():
    globals()['x'] += 5 # Access and modify the global variable using the dictionary

update_x()
print(x) # Output: 15
```

15

In []: